

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ESPECIALIDAD DE INGENIERÍA INFORMÁTICA



Proyecto de Diseño y Desarrollo de Software
[1INF54-0982]

Estándar de Programación Web Frontend

Equipo: 6F

Integrantes:

- Gabriel Angel Quispe Cerron
- Roly Alvaro Huacre Samaniego
- Amira Paola Rodriguez Almora
- Diego Alfredo Carmona Chumbimuni

2026-2

Lima, 15 de septiembre

Historial de versiones

Fecha	Versión	Descripción	Autor
15/09/2026	1.0	Se ha redactado la primera versión de los estándares de programación web frontend	Equipo 6F

Contenido

1. Introducción.....	3
2. Guías de estilo HTML.....	4
3. Guías de estilo CSS.....	4
4. Framework JavaScript/TypeScript (React).....	4
4.1. TypeScript.....	4
4.2. Componentes.....	5
5. Estructura de componentes.....	5
6. Manejo del estado global.....	6
7. Reglas de linter y formateo.....	6
8. Referencias y recursos.....	6

1. Introducción

Este documento define los estándares de programación web (HTML, CSS y el framework JavaScript/TypeScript) para el frontend del sistema SysMile. El frontend implementa el visualizador de mapa interactivo que muestra en tiempo real las rutas calculadas por el planificador, el estado de los vehículos y los pedidos asignados.

El stack tecnológico del frontend es React 18 con TypeScript, Vite como bundler y una librería de mapas (Leaflet o Mapbox GL) para la capa geoespacial.

2. Guías de estilo HTML

- Usar etiquetas semánticas (header, main, nav, section, article) en lugar de div genéricos cuando exista una etiqueta semántica adecuada.
- Todo elemento interactivo (botón, enlace) debe ser accesible por teclado y contar con atributos ARIA cuando su función no sea evidente por la etiqueta HTML.
- Los atributos deben ir en el orden: id, class, atributos de datos (data-*), atributos ARIA, eventos.
- Usar comillas dobles para los valores de atributos HTML.

2.1. Accesibilidad Avanzada y Contexto Semántico

- Para optimizar la interpretación de contexto de la interfaz, especialmente al interactuar con lectores de pantalla de código abierto o modelos de lenguaje semánticos locales, se debe mantener una jerarquía estricta en el árbol del DOM que exponga atributos ARIA descriptivos y precisos.
- Utilizar regiones con el atributo aria-live="polite" o aria-live="assertive" para anunciar dinámicamente los cambios de estado del sistema en tiempo real (por ejemplo, notificaciones del tipo "El vehículo 3 ha cambiado su ruta" o "Nuevo pedido asignado") sin requerir que el usuario pierda el foco de su tarea actual en el mapa.

3. Guías de estilo CSS

- Usar la metodología BEM (Block__Element--Modifier) para nombrar clases CSS en los módulos que no usan CSS-in-JS ni utilidades de Tailwind.
- Centralizar variables de diseño (colores, espaciados, tipografías) en variables CSS (custom properties) definidas en un archivo styles/tokens.css.
- Evitar selectores anidados de más de 3 niveles y el uso de !important salvo para sobrescribir estilos de librerías de terceros.

CSS

```
.route-card { padding: var(--spacing-md); }  
.route-card__title { font-weight: 600; }  
.route-card--active { border-color: var(--color-success); }
```

4. Framework JavaScript/TypeScript (React)

4.1. TypeScript

- Todo archivo nuevo debe escribirse en TypeScript (.ts / .tsx); no se permite JavaScript sin tipar en código nuevo.
- Definir interfaces o types para toda estructura de datos que provenga del backend (Route, Vehicle, Driver, Order), ubicadas en src/types/.
- Evitar el tipo any; usar unknown y realizar el afinado de tipo (type narrowing) cuando el tipo no se conozca de antemano.
- Habilitar strict: true en tsconfig.json.

4.2. Componentes

- Usar componentes funcionales con Hooks; no se permiten componentes de clase en código nuevo.
- Nombrar los componentes en PascalCase y ubicar cada uno en su propio archivo (RouteMap.tsx, VehicleMarker.tsx).
- Los componentes de presentación (UI pura) deben separarse de los componentes contenedores (que gestionan datos y estado) para facilitar las pruebas.

TypeScript

```
interface VehicleMarkerProps {  
  vehicle: Vehicle;  
  onSelect: (vehicleId: string) => void;  
}  
  
export function VehicleMarker({ vehicle, onSelect }:  
VehicleMarkerProps) {  
  return <Marker position={vehicle.position} onClick={() =>  
onSelect(vehicle.id)} />;  
}
```

5. Estructura de componentes

4.3. Convenciones de Nomenclatura para Eventos

- Utilizar el prefijo handle para nombrar las funciones internas de un componente que actúan como manejadores de eventos (ej. handleVehicleSelect, handleRouteSubmit).
- Utilizar el prefijo on para nombrar las propiedades (props) expuestas por los componentes de presentación que emiten eventos hacia sus componentes contenedores (ej. onSelect, onSubmit).

4.4. Rendimiento y Memoización (Optimización de Mapas)

- Utilizar `React.memo` exclusivamente en componentes puros de presentación que reciben propiedades complejas y corren el riesgo de renderizarse con alta frecuencia debido a las actualizaciones por sockets (ej. marcadores del mapa, polígonos de rutas o tarjetas emergentes).
- Envolver en `useMemo` los cálculos pesados derivados del estado global (como el filtrado espacial de cientos de pedidos) y usar `useCallback` para estabilizar las funciones que se pasen como dependencias a los efectos (`useEffect`) vinculados a la instancia del mapa.

4.5. Manejo de Errores Visuales (Error Boundaries)

- Es obligatorio implementar `Error Boundaries` (Límites de Error) alrededor de componentes críticos y propensos a fallos por asincronía o datos de terceros. Específicamente, el contenedor principal de la librería del mapa debe estar protegido para evitar que la aplicación colapse y muestre una pantalla en blanco si recibe coordenadas nulas o inválidas.
- Estandarizar el despliegue de los errores de red, gestionados a través de la librería de fetching, mediante un sistema centralizado de notificaciones (`Toasts` o `Snackbars`) no intrusivas.

Carpeta	Contenido
components/	Componentes reutilizables de UI (<code>RouteMap</code> , <code>VehicleMarker</code> , <code>StatusBadge</code>)
pages/	Vistas principales (<code>Dashboard</code> , <code>Rutas</code> , <code>Seguimiento en Vivo</code>)
features/	Módulos por dominio funcional (rutas, vehículos, pedidos), cada uno con sus componentes, hooks y servicios propios
hooks/	Hooks personalizados reutilizables entre features
services/	Clientes HTTP y lógica de comunicación con la API del backend

store/	Definición del estado global (contextos o stores de Zustand)
types/	Interfaces y tipos TypeScript compartidos

6. Manejo del estado global

- Usar useState para estado local de un componente (formularios, filtros de una vista).
- Usar Zustand como store principal para el estado global del mapa (ruta seleccionada, capa activa, posición de vehículos en tiempo real), evitando el prop drilling entre componentes anidados.
- Usar React Context únicamente para datos de configuración que cambian con poca frecuencia (usuario autenticado, tema, idioma).
- Los datos que provienen del servidor (rutas, pedidos, vehículos) deben gestionarse con una librería de fetching con caché (React Query / TanStack Query), no almacenarse manualmente en el store global.
- Toda actualización de estado en tiempo real (sockets de tracking de vehículos) debe despacharse a través de acciones del store, nunca mutando el estado directamente.

7. Enrutamiento y Navegación (Routing)

- Todas las rutas de la aplicación deben definirse de forma declarativa y centralizada utilizando un enrutador estándar para React (ej. React Router v6+).
- Implementar un componente contenedor de "Ruta Protegida" (Protected Route / Auth Guard) que verifique la presencia y validez del token JWT en el cliente antes de renderizar cualquier vista del planificador. Si la validación falla, el usuario debe ser redirigido inmediatamente a la vista de autenticación.
- El paso de parámetros simples entre vistas (como el ID de un vehículo seleccionado) debe realizarse a través de parámetros en la URL (/:vehicleId) o mediante Query Parameters, aprovechando las utilidades del enrutador para su extracción segura.

8. Reglas de linter y formateo

- Configuración base: eslint:recommended + plugin:@typescript-eslint/recommended + plugin:react-hooks/recommended.
- Prettier gestiona el formato (2 espacios, comillas simples, punto y coma obligatorio, ancho de línea 100); ESLint no debe definir reglas de formato que dupliquen a Prettier.
- Regla obligatoria react-hooks/exhaustive-deps activada como error, dado que el visualizador de mapa depende fuertemente de efectos con suscripciones (sockets, listeners del mapa) que deben limpiarse correctamente.
- No permitir imports circulares entre features (regla import/no-cycle).

- Prohibir `console.log` en el código de producción (permitir `console.warn` y `console.error` mediante regla `no-console` con excepciones).

9. Referencias y recursos

Guía de estilo de Airbnb JavaScript: <https://github.com/airbnb/javascript>

Documentación oficial de React: <https://react.dev>

Documentación oficial de TypeScript: <https://www.typescriptlang.org/docs/>

Documentación de Leaflet: <https://leafletjs.com/reference.html>